Polymorphism ("Poly" means many, and "Morphs" means forms)

Polymorphism is the ability of an object to take on different forms. In Java, polymorphism refers to the ability of a class to provide different implementations of a method, depending on the type of object that is passed to the method.

- Polymorphism in Java allows us to perform the same action in many ways.
- With polymorphism, we can design and implement systems that are easily extensible.
- You can perform Polymorphism in Java via two different methods: **Method Overloading** and **Method Overriding**.

Abstract Classes and Methods

Data abstraction is the process of hiding certain details and showing only essential information to the user. Abstraction can be achieved with either abstract classes or interfaces

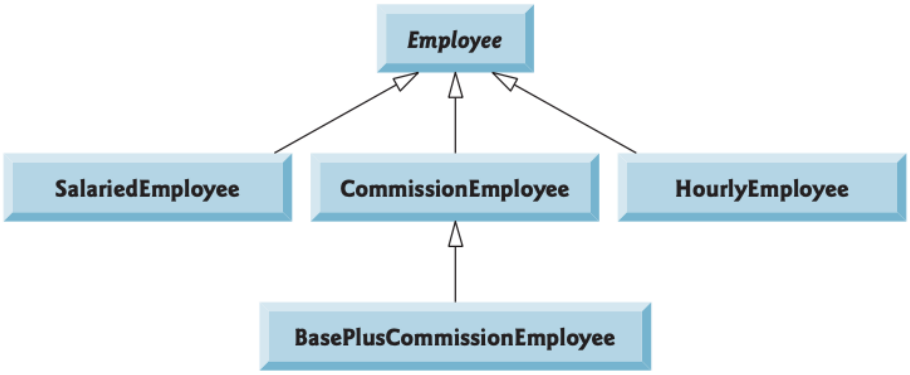
- Abstract classes cannot be used to instantiate objects because they're incomplete.
- The primary purpose of an abstract class is to provide an appropriate superclass from which other classes can inherit and thus share a common design.

The abstract keyword is a non-access modifier, used for classes and methods:

- **Abstract class**: is a restricted class that cannot be used to create objects (to access it, it must be inherited from another class).

- **Abstract method**: can only be used in an abstract class, and it does not have a body. The body is provided by the subclass (inherited from).

Example,



Employee.java | SalariedEmployee.java | HourlyEmployee.java | PayrollSystemTest.java

```
1  package abstractpolymorphism;
2
3  public class PayrollSystemTest {
4
5      public static void main( String[] args ) {
6
7      SalariedEmployee salariedEmployee =
8              new SalariedEmployee( "John", "Smith", "111-11-1111", 800.00 );
9
10     HourlyEmployee hourlyEmployee =
11             new HourlyEmployee( "Karen", "Price", "222-22-2222", 16.75, 40 );
12
13     System.out.println(salariedEmployee.getInfo());
14     System.out.println(hourlyEmployee.getInfo());
15     }
16 }
```

Employee.java | SalariedEmployee.java | HourlyEmployee.java | PayrollSystemTest.java

```java
package abstractpolymorphism;

public abstract class Employee {

private String firstName;
private String lastName;
private String socialSecurityNumber;

// three-argument constructor
public Employee( String first, String last, String ssn ){
   firstName = first;
   lastName = last;
   socialSecurityNumber = ssn;
} // end three-argument Employee constructor

// set first name
public void setFirstName( String first ){
firstName = first; // should validate
} // end method setFirstName

// return first name
public String getFirstName(){
return firstName;
} // end method getFirstName

// set last name
public void setLastName( String last ){
lastName = last; // should validate
} // end method setLastName

// return last name
public String getLastName(){
return lastName;
} // end method getLastName

// set social security number
public void setSocialSecurityNumber( String ssn ){
socialSecurityNumber = ssn; // should validate
} // end method setSocialSecurityNumber

// return social security number
public String getSocialSecurityNumber(){
return socialSecurityNumber;
} // end method getSocialSecurityNumber

// abstract method overridden by concrete subclasses
public abstract double earnings(); // no implementation here
} // end abstract class Employee
```

Employee.java | SalariedEmployee.java | HourlyEmployee.java | PayrollSystemTest.java

```java
package abstractpolymorphism;

// SalariedEmployee class extends Employee.
public class SalariedEmployee extends Employee{
 private double weeklySalary;

 // four-argument constructor
public SalariedEmployee( String first, String last, String ssn, double salary ){
    super( first, last, ssn ); // pass to Employee constructor
    setWeeklySalary( salary ); // validate and store salary
} // end four-argument SalariedEmployee constructor

// set salary
public void setWeeklySalary( double salary ){
    if (salary > 0.0)
    weeklySalary = salary;
} // end method setWeeklySalary

// return salary
public double getWeeklySalary(){
 return weeklySalary;
} // end method getWeeklySalary

// calculate earnings; override abstract method earnings in Employee
public double earnings() {
    return getWeeklySalary();
} // end method earnings

public String getInfo(){
     return (getFirstName() + " " +  getLastName() + "\n" + getSocialSecurityNumber() + "\n" +
             earnings());
     } //
} // end class SalariedEmployee
```

Employee.java | SalariedEmployee.java | *HourlyEmployee.java | PayrollSystemTest.java

```java
package abstractpolymorphism;
// HourlyEmployee class extends Employee.

public class HourlyEmployee extends Employee{
private double wage; // wage per hour
private double hours; // hours worked for week

// five-argument constructor
public HourlyEmployee( String first, String last, String ssn, double hourlyWage, double hoursWorked ){
super( first, last, ssn );
setWage( hourlyWage); // validate hourly wage
setHours( hoursWorked); // validate hourly worked
} // end five-argument HourlyEmployee constructor

// set wage
public void setWage( double hourlyWage){
    if ( hourlyWage >= 0.0 )
        wage = hourlyWage;
} // end method setWage

//set hours worked
public void setHours( double hoursWorked ){
    if ( ( hoursWorked >= 0.0 ) && ( hoursWorked <= 168.0 ) )
        hours = hoursWorked;
    } // end method setHours

public double earnings() {
        return wage * hours;
} // end method earnings

public String getInfo(){
     return (getFirstName() + " " +  getLastName() + "\n" + getSocialSecurityNumber() + "\n" +
             earnings());
     } //
} // end class HourlyEmployee
```

Problems | Javadoc | Declaration | Console | Progress

```
<terminated> PayrollSystemTest [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_112.jdk/C
John Smith
111-11-1111
800.0
Karen Price
222-22-2222
670.0
```

Homework

Q> Using programming show the effect of using ***final*** with Methods and Classes.